

NumPy for Python

A 3-Hour Instructor-Led Training

Instructor & Participant Training Manual

From Arrays to Linear Algebra — A Practical, Hands-On Guide

Detail	Description
Duration	3 Hours (Instructor-Led)
Audience	Students, Developers, Data Analysts, Python Beginners
Prerequisites	Basic Python programming knowledge
Format	Lecture + Live Coding + Hands-on Exercises + Quiz
Version Covered	NumPy 1.26+ / 2.x

Table of Contents

Table of Contents	2
About This Training	6
Target Audience	6
Learning Objectives	6
Training Agenda	6
1. Introduction	8
1.1 What is NumPy?	8
1.2 Why NumPy?	8
1.3 Advantages Over Python Lists	8
1.4 NumPy Architecture	8
1.5 Installation	9
1.6 Importing NumPy	9
2. NumPy Arrays	10
2.1 Creating Arrays	10
1D Arrays	10
2D Arrays	10
3D Arrays	10
Other Array Creation Functions	11
2.2 Data Types (dtype)	11
2.3 Shape, Dimensions, and Size	11
2.4 Reshaping Arrays	12
2.5 Flattening Arrays	12
3. Array Operations	13
3.1 Arithmetic Operators	13
3.2 Comparison Operators	13
3.3 Universal Functions (ufuncs)	13
4. Indexing and Slicing	15
4.1 Basic Indexing	15
4.2 Negative Indexing	15
4.3 Slicing	15
4.4 Step Slicing	15
4.5 Fancy Indexing	16
4.6 Boolean Indexing / Filtering	16

5. Mathematical Functions	17
6. Array Manipulation	18
6.1 Concatenate & Stack	18
6.2 Split	18
6.3 Append, Insert, Delete	18
6.4 Sort, Unique, Reverse	18
7. Broadcasting	19
7.1 Broadcasting Rules	19
7.2 Real-World Example	19
7.3 Advantages of Broadcasting	19
7.4 Common Interview Questions on Broadcasting	20
8. Matrix Operations	21
8.1 Matrix Creation	21
8.2 Matrix Multiplication	22
8.3 Transpose	22
8.4 Inverse & Determinant	22
8.5 Eigenvalues & Eigenvectors	23
8.6 Dot Product	23
9. Random Module	24
9.1 Random Integers	25
9.2 Random Floats	25
9.3 Normal (Gaussian) Distribution	26
9.4 Uniform Distribution	26
9.5 Seed	26
9.6 Shuffle	27
9.7 Choice	27
10. Performance Comparison	28
Execution Time & Memory Usage	29
11. Practical Hands-on Demo (40 Minutes)	30
Project: Student Marks Analysis System	30
Step 1 — Dataset Creation	30
Step 2 — Average Marks per Student	30
Step 3 — Highest and Lowest Marks	30
Step 4 — Standard Deviation per Student	30
Step 5 — Ranking Students	31

Step 6 — Top Students (Above Class Average).....	31
Step 7 — Subject-wise Analysis	31
Step 8 — Percentage Calculation.....	31
Step 9 — Visualization-Ready Data.....	31
12. Mini Exercises.....	33
12.1 Practice Questions (20) — Conceptual	33
12.2 Coding Exercises (10)	34
Exercise 1 — Create and Inspect.....	34
Exercise 2 — Reshape	34
Exercise 3 — Row and Column Sums.....	34
Exercise 4 — Boolean Filtering.....	34
Exercise 5 — Normalize an Array.....	35
Exercise 6 — Matrix Multiplication.....	35
Exercise 7 — Random Sampling.....	35
Exercise 8 — Broadcasting Practice	35
Exercise 9 — Statistics Summary	35
Exercise 10 — Ranking	36
12.3 Debugging Exercises (5)	36
Debug 1	36
Debug 2	36
Debug 3	36
Debug 4	37
Debug 5	37
13. Interview Questions.....	38
13.1 Beginner	38
13.2 Intermediate	38
13.3 Advanced.....	39
14. Best Practices	41
14.1 Memory Optimization.....	41
14.2 Vectorization.....	41
14.3 Broadcasting	41
14.4 Common Mistakes	41
14.5 Performance Tips	41
15. Cheat Sheet — Quick Reference.....	43
Array Creation.....	43

Inspecting Arrays.....	43
Mathematical Functions	43
Indexing & Slicing	44
Broadcasting & Matrix Ops	44
Random Module	44
16. Summary	45
Where to Go Next	45

© Shalinee Priya

About This Training

This manual is designed for a single, focused, 3-hour classroom or virtual session introducing NumPy, the foundational numerical computing library for Python. It is written to serve two audiences at once: the instructor delivering the material, and the participant who will keep the manual afterward as a reference.

Target Audience

- Students studying programming, data science, or engineering
- Software developers moving into data-heavy or scientific work
- Data analysts who currently rely on spreadsheets or pandas without NumPy fundamentals
- Any beginner with basic Python knowledge (variables, loops, functions, lists)

Learning Objectives

By the end of this session, participants will be able to:

- Understand what NumPy is and why it is used
- Create and manipulate NumPy arrays
- Perform mathematical and statistical operations
- Use indexing, slicing, and boolean masking
- Understand array broadcasting
- Work with multidimensional arrays
- Perform matrix operations and linear algebra
- Generate random data
- Optimize Python programs using NumPy
- Apply NumPy in a practical data science scenario

Training Agenda

#	Topic	Duration
1	Introduction	10 min
2	NumPy Basics (Arrays)	30 min
3	Array Operations	35 min
4	Mathematical Functions	20 min
5	Indexing & Slicing	20 min
6	Broadcasting & Advanced Operations	25 min
7	Matrix Operations & Linear Algebra	20 min
8	Practical Hands-on Demo	40 min

#	Topic	Duration
9	Quiz & Q&A	20 min
	Total	3 hr 0 min

◆ TIP

Timings are approximate. Build in a 5–10 minute buffer around the halfway point for a short break during a live session.

© Shalinee Priya

1. Introduction

1.1 What is NumPy?

NumPy (Numerical Python) is the core open-source library for numerical and scientific computing in Python. It provides a fast, memory-efficient multidimensional array object called `ndarray`, along with a large collection of high-level mathematical functions to operate on these arrays.

Nearly every major data science and machine learning library in Python — pandas, scikit-learn, TensorFlow, PyTorch, Matplotlib — is built on top of NumPy or is designed to interoperate with it. Learning NumPy well is a prerequisite for effective data work in Python.

1.2 Why NumPy?

Python's built-in lists are flexible but slow for numerical work, because each element is a full Python object with its own type and memory overhead. NumPy solves this by storing data in contiguous blocks of memory, using a fixed data type per array, and pushing loops down into compiled C code.

1.3 Advantages Over Python Lists

Feature	Python List	NumPy Array
Memory layout	Scattered objects, high overhead	Contiguous, fixed-type, compact
Speed	Slow (interpreted loops)	Fast (vectorized, C-compiled loops)
Data types	Mixed types allowed	Single, fixed dtype per array
Math operations	Manual loops / list comprehensions	Built-in vectorized operators
Broadcasting	Not supported	Fully supported
Functionality	General purpose	Linear algebra, stats, FFT, random, etc.

◆ TIP

Rule of thumb: if you find yourself writing a for-loop over numeric data, ask whether a NumPy vectorized operation could replace it.

1.4 NumPy Architecture

At the center of NumPy is the `ndarray` — a homogeneous, N-dimensional grid of values, of all the same dtype, described by a shape (tuple of dimension sizes) and strides (how many bytes to step in memory to move along each dimension). This is illustrated below.

CODE

ndarray object

data buffer (contiguous memory)	
dtype (e.g. int32, float64)	
shape (e.g. (3, 4))	
strides (bytes per step per axis)	

Above this core object, NumPy layers modules for linear algebra (`numpy.linalg`), random number generation (`numpy.random`), Fourier transforms (`numpy.fft`), and general mathematical functions (universal functions, or “ufuncs”).

1.5 Installation

CODE

```
# Using pip
pip install numpy

# Using conda
conda install numpy

# Verify installation
python -c "import numpy; print(numpy.__version__)"
```

1.6 Importing NumPy

By convention, NumPy is always imported with the alias `np`. This convention is used throughout the data science ecosystem and should be followed consistently.

CODE

```
import numpy as np

print(np.__version__)
```

OUTPUT

2.1.0

NOTE

Every code example in this manual assumes `import numpy as np` has already been run.

2. NumPy Arrays

2.1 Creating Arrays

1D Arrays

CODE

```
a = np.array([1, 2, 3, 4, 5])
print(a)
print(type(a))
```

OUTPUT

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

2D Arrays

CODE

```
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b)
```

OUTPUT

```
[[1 2 3]
 [4 5 6]]
```

3D Arrays

CODE

```
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(c)
print(c.ndim)
```

OUTPUT

```
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
3
```

Other Array Creation Functions

CODE

```
np.zeros((2, 3))      # array of zeros, shape (2,3)
np.ones((3, 3))      # array of ones
np.full((2, 2), 7)    # array filled with 7
np.eye(3)             # 3x3 identity matrix
np.arange(0, 10, 2)   # [0 2 4 6 8]
np.linspace(0, 1, 5)  # 5 evenly spaced values from 0 to 1
```

2.2 Data Types (dtype)

Every NumPy array has a single dtype shared by all elements. NumPy infers the dtype automatically, but it can be set explicitly with the dtype argument.

CODE

```
a = np.array([1, 2, 3])
print(a.dtype)          # int64

b = np.array([1, 2, 3], dtype=np.float32)
print(b.dtype)          # float32

c = a.astype(np.float64) # convert dtype
```

dtype	Description
int8 / int16 / int32 / int64	Signed integers of various widths
uint8 / uint16 / uint32 / uint64	Unsigned integers
float32 / float64	Single / double precision floating point
bool	Boolean values True/False
complex64 / complex128	Complex numbers

2.3 Shape, Dimensions, and Size

CODE

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape)  # (2, 3) -> 2 rows, 3 columns
print(arr.ndim)   # 2      -> number of dimensions
print(arr.size)   # 6      -> total number of elements
print(arr.dtype)  # int64
```

OUTPUT

```
(2, 3)
2
```

```
6
int64
```

2.4 Reshaping Arrays

`reshape()` changes the shape of an array without changing its data, as long as the total number of elements stays the same.

CODE

```
a = np.arange(12)          # [0 1 2 ... 11]
b = a.reshape(3, 4)
print(b)

c = a.reshape(2, -1)       # -1 lets NumPy infer that dimension
print(c.shape)             # (2, 6)
```

OUTPUT

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
(2, 6)
```

2.5 Flattening Arrays

CODE

```
arr = np.array([[1, 2], [3, 4]])
print(arr.flatten())  # returns a copy
print(arr.ravel())    # returns a view when possible
```

OUTPUT

```
[1 2 3 4]
[1 2 3 4]
```

✦ TIP

`flatten()` always returns a new copy; `ravel()` returns a view whenever it can, making it slightly faster and more memory-efficient.

3. Array Operations

NumPy overloads Python's standard operators so that arithmetic on arrays happens element-by-element (“element-wise”), without writing an explicit loop.

3.1 Arithmetic Operators

CODE

```
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])

print(a + b)    # Addition      -> [11 22 33 44]
print(a - b)    # Subtraction   -> [ 9 18 27 36]
print(a * b)    # Multiplication -> [10 40 90 160]
print(a / b)    # Division      -> [10. 10. 10. 10.]
print(a % b)    # Modulus       -> [0 0 0 0]
print(a ** 2)   # Exponent      -> [100 400 900 1600]
```

3.2 Comparison Operators

CODE

```
a = np.array([1, 2, 3, 4])
b = np.array([4, 3, 2, 1])
print(a > b)    # [False False True True]
print(a == b)   # [False False False False]
print(a <= 2)   # [ True  True False False]
```

OUTPUT

```
[False False True True]
[False False False False]
[ True  True False False]
```

3.3 Universal Functions (ufuncs)

A ufunc is a function that operates element-wise on arrays, implemented in compiled C for speed. Arithmetic operators are themselves shorthand for ufuncs ($a + b$ calls `np.add(a, b)`).

CODE

```
np.add(a, b)
np.subtract(a, b)
np.multiply(a, b)
np.divide(a, b)
np.power(a, 2)
np.mod(a, b)
np.abs(np.array([-1, -2, 3])) # [1 2 3]
```

✦ TIP

Ufuncs support broadcasting, type casting, and optional output arrays (`out=` parameter) for in-place computation — all of which make them faster and more memory-efficient than manual loops.

© Shalinee Priya

4. Indexing and Slicing

4.1 Basic Indexing

CODE

```
a = np.array([10, 20, 30, 40, 50])
print(a[0])      # 10 (first element)
print(a[3])      # 40 (fourth element)

b = np.array([[1, 2, 3], [4, 5, 6]])
print(b[1, 2])   # 6 (row 1, column 2)
```

4.2 Negative Indexing

CODE

```
a = np.array([10, 20, 30, 40, 50])
print(a[-1])     # 50 (last element)
print(a[-2])     # 40 (second-to-last)
```

4.3 Slicing

CODE

```
a = np.array([10, 20, 30, 40, 50])
print(a[1:4])    # [20 30 40]
print(a[:3])     # [10 20 30]
print(a[2:])     # [30 40 50]
```

4.4 Step Slicing

CODE

```
a = np.arange(10)
print(a[::2])    # [0 2 4 6 8]    every 2nd element
print(a[1:8:2])  # [1 3 5 7]
print(a[::-1])   # reverse the array
```

OUTPUT

```
[0 2 4 6 8]
[1 3 5 7]
[9 8 7 6 5 4 3 2 1 0]
```

4.5 Fancy Indexing

Fancy indexing selects elements using a list or array of indices, rather than a single index or slice. Unlike basic slicing, it always returns a copy, not a view.

CODE

```
a = np.array([10, 20, 30, 40, 50])
idx = [0, 2, 4]
print(a[idx])    # [10 30 50]
```

4.6 Boolean Indexing / Filtering

CODE

```
a = np.array([10, 15, 20, 25, 30])
mask = a > 18
print(mask)      # [False False  True  True  True]
print(a[mask])   # [20 25 30]
print(a[a % 10 == 0]) # [10 20 30]
```

OUTPUT

```
[False False  True  True  True]
[20 25 30]
[10 20 30]
```

⚠ WARNING

Basic slices (`a[1:4]`) return views that share memory with the original array; modifying the slice modifies the original. Fancy and boolean indexing return copies.

5. Mathematical Functions

CODE

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

Function	Description	Example Result
<code>np.sum(a)</code>	Sum of all elements	55
<code>np.mean(a)</code>	Arithmetic mean	5.5
<code>np.median(a)</code>	Median value	5.5
<code>np.std(a)</code>	Standard deviation	2.872...
<code>np.var(a)</code>	Variance	8.25
<code>np.min(a)</code>	Minimum value	1
<code>np.max(a)</code>	Maximum value	10
<code>np.percentile(a, 75)</code>	75th percentile	7.75
<code>np.sqrt(a)</code>	Element-wise square root	[1. 1.41 ... 3.16]
<code>np.log(a)</code>	Natural logarithm	[0. 0.69 ... 2.30]
<code>np.exp(a)</code>	Exponential (e^x)	[2.71 7.38 ...]
<code>np.sin(a)</code>	Sine (radians)	[0.84 0.90 ...]

◆ TIP

`np.std()` and `np.var()` use a population formula (divide by N) by default. Pass `ddof=1` for the sample formula (divide by N-1), matching pandas' default.

6. Array Manipulation

6.1 Concatenate & Stack

CODE

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.concatenate([a, b])      # [1 2 3 4 5 6]
np.vstack([a, b])           # stack as rows -> shape (2,3)
np.hstack([a, b])           # stack horizontally -> [1 2 3 4 5 6]
np.stack([a, b], axis=0)    # new axis -> shape (2,3)
```

6.2 Split

CODE

```
a = np.arange(9)
np.split(a, 3)              # [array([0,1,2]), array([3,4,5]), array([6,7,8])]
```

6.3 Append, Insert, Delete

CODE

```
a = np.array([1, 2, 3])
np.append(a, [4, 5])        # [1 2 3 4 5]
np.insert(a, 1, 99)         # [1 99 2 3]
np.delete(a, 0)             # [2 3]
```

6.4 Sort, Unique, Reverse

CODE

```
a = np.array([3, 1, 4, 1, 5, 9, 2])
np.sort(a)                  # [1 1 2 3 4 5 9]
np.unique(a)                # [1 2 3 4 5 9]
a[::-1]                    # reverse -> [2 9 5 1 4 1 3]
```

7. Broadcasting

Broadcasting is the set of rules NumPy uses to perform arithmetic on arrays of different shapes without making unnecessary copies of data. It lets you write `a * 2` instead of looping over every element of `a`.

7.1 Broadcasting Rules

When comparing the shapes of two arrays element-by-element from the rightmost dimension:

- If the dimensions are equal, they are compatible.
- If one of the dimensions is 1, it is stretched to match the other.
- If the dimensions differ and neither is 1, the operation fails with a `ValueError`.

CODE

```
Shape A:  (3, 4)
Shape B:   (4,)
Result:  (3, 4)  <- B is broadcast across each of the 3 rows
```

7.2 Real-World Example

CODE

```
prices = np.array([100, 200, 300])  # shape (3,)
tax_rate = 1.18                      # scalar, shape ()
final_prices = prices * tax_rate
print(final_prices)
```

OUTPUT

```
[118. 236. 354.]
```

CODE

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])  # shape (2, 3)
row_add = np.array([10, 20, 30])          # shape (3,)
print(matrix + row_add)
```

OUTPUT

```
[[11 22 33]
 [14 25 36]]
```

7.3 Advantages of Broadcasting

- Avoids explicit loops — faster and more readable
- Avoids creating unnecessary intermediate copies of data

- Makes code that works across scalars, vectors, and matrices uniform

7.4 Common Interview Questions on Broadcasting

- What are the two rules that decide whether two shapes can broadcast?
- Can a (3,1) array broadcast with a (1,4) array? What is the resulting shape? (Yes — result is (3,4))
- Why is broadcasting faster than manually tiling an array with `np.tile` before an operation?

© Shalinee Priya

8. Matrix Operations

Why Do We Study Matrix Operations?

Matrices are used everywhere.

- Artificial Intelligence
- Machine Learning
- Computer Vision
- Image Processing
- Data Science
- Statistics
- Robotics
- Games
- Engineering

Almost every ML algorithm internally performs millions of matrix operations.

8.1 Matrix Creation

CODE

```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])
```

The 12 Most Important Methods (Must Learn)

These are the methods every Data Analyst or Data Scientist uses regularly:

1. `np.array()`
2. `np.zeros()`
3. `np.ones()`
4. `np.full()`
5. `np.eye()`
6. `np.diag()`
7. `np.arange().reshape()`
8. `np.linspace().reshape()`
9. `np.random.rand()`
10. `np.random.randint()`
11. `np.vstack()` / `np.hstack()`
12. `np.concatenate()`

8.2 Matrix Multiplication

CODE

```
np.dot(A, B)      # matrix (dot) product
A @ B            # equivalent, preferred modern syntax
```

OUTPUT

```
[[19 22]
 [43 50]]
```

⚠ WARNING

`A * B` performs element-wise multiplication, NOT matrix multiplication. Use `A @ B` or `np.dot(A, B)` for true matrix multiplication.

8.3 Transpose

CODE

```
A.T
np.transpose(A)
```

OUTPUT

```
[[1 3]
 [2 4]]
```

Uses

- Image processing
- Statistics
- Machine Learning
- Matrix multiplication

8.4 Inverse & Determinant

These require a square matrix (same number of rows and columns).

Determinant

The determinant is a single number that tells us important properties of a matrix.

For example:

```
A = np.array([[2,3],
              [1,4]])
```

Inverse

Inverse is like reciprocal.

Property: $A \times A^{-1} = I$ (Identity Matrix)

CODE

```
np.linalg.det(A)    # determinant -> -2.0
np.linalg.inv(A)    # matrix inverse
```

8.5 Eigenvalues & Eigenvectors

This is one of the most important concepts in Machine Learning. Imagine you stretch a rubber sheet. Most arrows change both length and direction. But some special arrows keep the same direction; only their length changes.

Those special arrows are eigenvectors. The stretching amount is the eigenvalue.

Mathematically:

CODE

```
values, vectors = np.linalg.eig(A)
print(values)
print(vectors)
```

Real-world Uses

- Google PageRank
- Face Recognition
- PCA (Principal Component Analysis)
- Recommendation Systems
- Image Compression

8.6 Dot Product

CODE

```
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])
np.dot(v1, v2)    # 1*4 + 2*5 + 3*6 = 32
```

Interview Questions

1. What is the difference between a matrix and a NumPy array?
 2. What is the difference between $A * B$ and $A @ B$?
 3. What is an identity matrix?
 4. What does the transpose of a matrix do?
 5. When does a matrix have an inverse?
 6. What does a determinant tell us?
 7. What are eigenvalues and eigenvectors in simple terms?
 8. What is the difference between `np.dot()` and `np.matmul()`?
 9. What happens if you try to multiply incompatible matrices?
 10. Where are matrix operations used in real-world applications?
-

9. Random Module

`numpy.random` provides tools for generating pseudo-random numbers, used constantly for testing code, simulations, games, Password Generation, statistics, initializing machine learning models, AI, and sampling data.

NOTE

Modern NumPy code should prefer the Generator API (`np.random.default_rng()`) over the older global-state functions shown for legacy familiarity below.

Earlier versions of NumPy used functions like `np.random.randint()` and `np.random.seed()`. Modern NumPy recommends creating a random number generator with `np.random.default_rng()` and then calling methods such as `integers()`, `random()`, `normal()`, and `choice()`.

Recommended (Modern) Approach

Here, `rng` is an independent **Random Number Generator (RNG)** object. All random operations are performed using this object.

9.1 Random Integers

CODE

```
rng = np.random.default_rng(seed=42)
or,
rng = np.random.default_rng(42)
rng.integers(low=1, high=100, size=5)
rng.integers(1, 50, size=(3,4))
```

Parameter	Meaning
<code>low</code>	Starting value (inclusive)
<code>high</code>	Ending value (exclusive)
<code>size</code>	Number or shape of random values

9.2 Random Floats

Generates random floating-point numbers uniformly distributed between 0 (inclusive) and 1 (exclusive).

CODE

```
rng.random(5)           # 5 floats in [0, 1)
rng.random((2,3))       # to generate a matrix
```

9.3 Normal (Gaussian) Distribution

Generates random numbers following a Normal (Bell-Shaped) Distribution.

Many real-world data such as heights, weights, exam scores, and measurement errors approximately follow this distribution.

CODE

```
rng.normal(loc=0, scale=1, size=5)  # mean 0, std 1
```

Parameters

- **loc** → Mean
- **scale** → Standard Deviation
- **size** → Number or shape of values

9.4 Uniform Distribution

Generates random floating-point numbers where every value has an equal probability of occurring within the specified range.

CODE

```
rng.uniform(low=0, high=10, size=5)
```

Parameters

- **low** → Lower limit (inclusive)
- **high** → Upper limit (exclusive)
- **size** → Number or shape of values

9.5 Seed

A seed makes random generation reproducible, the same seed always produces the same sequence, which is essential for debugging, testing, Research, ML and reproducible experiments.

CODE

```
rng = np.random.default_rng(seed=42)
print(rng.integers(0, 10, 3))          # same output every run
```

Note: Unlike the older API (`np.random.seed()`), the Generator API sets the seed while creating the generator object.

9.6 Shuffle

Randomly rearranges the elements of an array in place.

```
CODE
a = np.arange(10)
rng.shuffle(a)    # shuffles a in place
print(a)
```

Note: shuffle() modifies the original array. It does not return a new array.

9.7 Choice

Randomly selects one or more values from an existing array.

```
CODE
options = np.array(['A', 'B', 'C', 'D'])
rng.choice(options, size=3, replace=False)
```

Parameters

- **array** → Input array
- **size** → Number of values to select
- **replace** → Whether duplicate selections are allowed

Difference Between Uniform and Normal Distribution

Uniform Distribution

Equal probability for all values

Every value is equally likely

Used in simulations

Normal Distribution

Bell-shaped probability

Middle values are more likely

Used in statistics and Machine Learning

Legacy Functions (For Awareness)

Older NumPy code often uses the following functions:

Legacy API

`np.random.randint()`
`np.random.rand()`
`np.random.randn()`
`np.random.normal()`
`np.random.uniform()`
`np.random.seed()`
`np.random.shuffle()`
`np.random.choice()`

Modern Generator API

`rng.integers()`
`rng.random()`
`rng.normal()`
`rng.normal()`
`rng.uniform()`
`np.random.default_rng(seed)`
`rng.shuffle()`
`rng.choice()`

10. Performance Comparison

The single biggest practical reason to use NumPy is performance. The benchmark below compares summing one million numbers with a pure Python list versus a NumPy array.

CODE

```
import time
import numpy as np

size = 1_000_000
py_list = list(range(size))
np_array = np.arange(size)

start = time.time()
sum([x * 2 for x in py_list])
print('Python list:', time.time() - start, 'seconds')

start = time.time()
```

```
np_array * 2
print('NumPy array:', time.time() - start, 'seconds')
```

OUTPUT

Python list: 0.082134 seconds
NumPy array: 0.001745 seconds

Execution Time & Memory Usage

Metric	Python List	NumPy Array	Typical Improvement
Sum of 1M numbers	~0.08 sec	~0.002 sec	~40x faster
Memory (1M ints)	~28 MB	~8 MB	~3.5x less memory
Element access pattern	Pointer chasing	Contiguous block	Better CPU cache use

NumPy is faster for two main reasons:

- It performs operations using optimized C code instead of Python loops.
- It stores data compactly in a contiguous memory block, making memory access more efficient and computations faster while using less memory than scattered Python objects.

That's why Pandas, TensorFlow, PyTorch and almost every Data Science library is built on top of NumPy.

11. Practical Hands-on Demo (40 Minutes)

Project: Student Marks Analysis System

This walkthrough builds a small, realistic analysis pipeline using only NumPy.

Have participants type each step along with you; pause after each block to check output.

Step 1 — Dataset Creation

```
CODE
import numpy as np

students = ['Aditi', 'Rahul', 'Meera', 'Karan', 'Sana']
subjects = ['Math', 'Science', 'English']

# rows = students, columns = subjects
marks = np.array([
    [85, 78, 92],
    [67, 74, 69],
    [91, 88, 95],
    [55, 60, 58],
    [73, 81, 77],
])
print(marks.shape)    # (5, 3)
```

Step 2 — Average Marks per Student

```
CODE
avg_per_student = marks.mean(axis=1)
print(avg_per_student)
```

```
OUTPUT
[85.  70.  91.33  57.67  77.]
```

Step 3 — Highest and Lowest Marks

```
CODE
highest = marks.max()
lowest = marks.min()
print(highest, lowest)    # 95 55
```

Step 4 — Standard Deviation per Student

```
CODE
std_per_student = marks.std(axis=1)
```

```
print(std_per_student)
```

Step 5 — Ranking Students

CODE

```
ranking = np.argsort(-avg_per_student) # descending order of average
for rank, idx in enumerate(ranking, start=1):
    print(rank, students[idx], avg_per_student[idx])
```

OUTPUT

```
1 Meera 91.33
2 Aditi 85.0
3 Sana 77.0
4 Rahul 70.0
5 Karan 57.67
```

Step 6 — Top Students (Above Class Average)

CODE

```
class_average = avg_per_student.mean()
top_students = np.array(students)[avg_per_student > class_average]
print(top_students)
```

Step 7 — Subject-wise Analysis

CODE

```
subject_avg = marks.mean(axis=0)
for subj, avg in zip(subjects, subject_avg):
    print(subj, round(avg, 2))
```

Step 8 — Percentage Calculation

CODE

```
total_marks = marks.sum(axis=1)
percentage = (total_marks / (marks.shape[1] * 100)) * 100
print(percentage)
```

Step 9 — Visualization-Ready Data

The final step reshapes results into simple (label, value) pairs ready to hand to Matplotlib or pandas for charting — NumPy's job is to compute the numbers; a plotting library takes it from there.

CODE

```
chart_data = list(zip(students, avg_per_student))
print(chart_data)
# [('Aditi', 85.0), ('Rahul', 70.0), ('Meera', 91.33), ...]
```

◆ TIP

This entire pipeline — loading a table, computing per-row and per-column statistics, ranking, filtering, and preparing chart data — mirrors what happens inside pandas, which is itself built on NumPy.

© Shalinee Priya

12. Mini Exercises

Use these exercises for guided practice or as take-home reinforcement. Solutions are provided for self-study; instructors may wish to withhold them during live practice.

12.1 Practice Questions (20) — Conceptual

Q1. What function creates an array of zeros with shape (3,4)?

A: `np.zeros((3, 4))`

Q2. How do you check the number of dimensions of an array `a`?

A: `a.ndim`

Q3. How do you check the shape of an array `a`?

A: `a.shape`

Q4. How do you convert an array's dtype to float64?

A: `a.astype(np.float64)`

Q5. What does `np.arange(0, 10, 2)` return?

A: `[0 2 4 6 8]`

Q6. What is the difference between `np.zeros()` and `np.empty()`?

A: `np.empty()` allocates memory without initializing values (faster but contains garbage); `np.zeros()` initializes every element to 0.

Q7. How do you reverse a 1D array `a`?

A: `a[::-1]`

Q8. What does the -1 argument mean in `a.reshape(2, -1)`?

A: It tells NumPy to automatically infer that dimension's size based on the array's total element count.

Q9. How do you get every 3rd element of array `a`?

A: `a[::3]`

Q10. What is the result of `np.array([1,2,3]) + np.array([10,20,30])`?

A: `[11 22 33]`

Q11. How do you compute the transpose of matrix A?

A: `A.T` or `np.transpose(A)`

Q12. What is the difference between `a.flatten()` and `a.ravel()`?

A: `flatten()` always returns a copy; `ravel()` returns a view when possible, making it faster.

Q13. How do you find all elements of `a` greater than 10?

A: `a[a > 10]`

Q14. What does `np.linspace(0, 1, 5)` generate?

A: 5 evenly spaced values from 0 to 1 inclusive: `[0. 0.25 0.5 0.75 1.]`

Q15. How do you stack two 1D arrays as rows of a 2D array?

A: `np.vstack([a, b])`

Q16. What is broadcasting?

A: NumPy's rule set for performing arithmetic between arrays of different, compatible shapes without explicit copying or looping.

Q17. How do you compute the dot product of two vectors v1 and v2?

A: `np.dot(v1, v2)` or `v1 @ v2`

Q18. What does `a.sum(axis=0)` compute on a 2D array?

A: The sum down each column (collapsing the row axis).

Q19. How do you set a random seed with the modern Generator API?

A: `rng = np.random.default_rng(seed=42)`

Q20. How do you find the unique values in array `a`?

A: `np.unique(a)`

12.2 Coding Exercises (10)

Exercise 1 — Create and Inspect

Create a 1D array of the numbers 1 to 20. Print its shape, dtype, and sum.

CODE

```
a = np.arange(1, 21)
print(a.shape, a.dtype, a.sum())
```

Exercise 2 — Reshape

Reshape the array from Exercise 1 into a 4x5 matrix.

CODE

```
b = a.reshape(4, 5)
print(b)
```

Exercise 3 — Row and Column Sums

Using the 4x5 matrix `b`, compute the sum of each row and each column.

CODE

```
row_sums = b.sum(axis=1)
col_sums = b.sum(axis=0)
```

Exercise 4 — Boolean Filtering

From `b`, extract only the even numbers.

CODE

```
evens = b[b % 2 == 0]
print(evens)
```

Exercise 5 — Normalize an Array

Given `arr = np.array([2, 8, 4, 16])`, rescale it to the range `[0, 1]`.

CODE

```
arr = np.array([2, 8, 4, 16])
normalized = (arr - arr.min()) / (arr.max() - arr.min())
```

Exercise 6 — Matrix Multiplication

Multiply a 2x3 matrix by a 3x2 matrix using matrix multiplication.

CODE

```
A = np.array([[1,2,3],[4,5,6]])
B = np.array([[1,2],[3,4],[5,6]])
print(A @ B)
```

Exercise 7 — Random Sampling

Generate 10 random integers between 1 and 100 using a seeded generator, then sort them.

CODE

```
rng = np.random.default_rng(seed=1)
nums = rng.integers(1, 101, 10)
print(np.sort(nums))
```

Exercise 8 — Broadcasting Practice

Given a 3x3 matrix of prices and a 1D array of 3 discount multipliers (one per column), apply the discounts using broadcasting.

CODE

```
prices = np.array([[100,200,300]]*3)
discounts = np.array([0.9, 0.8, 0.95])
print(prices * discounts)
```

Exercise 9 — Statistics Summary

Given `scores = np.array([55,67,72,90,88,45,60])`, print the mean, median, std, min, and max.

CODE

```
scores = np.array([55,67,72,90,88,45,60])
print(scores.mean(), np.median(scores), scores.std(), scores.min(), scores.max())
```

Exercise 10 — Ranking

Given the scores from Exercise 9, print indices that would sort scores in descending order.

```
CODE
ranking = np.argsort(-scores)
print(ranking)
```

12.3 Debugging Exercises (5)

Each snippet below contains one bug. Identify and fix it.

Debug 1

```
CODE
a = np.array([1, 2, 3])
b = np.array([1, 2])
print(a + b)  # ValueError!
```

Issue: Shapes (3,) and (2,) are not broadcast-compatible — neither dimension is 1 and they are unequal.

Fix: Make the arrays the same length, or use an operation that expects mismatched shapes intentionally (e.g. pad b to length 3).

Debug 2

```
CODE
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
result = A * B  # expected matrix product
```

Issue: `\*` performs element-wise multiplication, not matrix multiplication.

Fix: Use `A @ B` or `np.dot(A, B)` instead.

Debug 3

```
CODE
a = np.arange(10)
view = a[2:5]
view[0] = 99
print(a)  # surprised a changed
```

Issue: This isn't actually a bug — it's a common misunderstanding: basic slices are views, not copies, so editing `view` edits `a`.

Fix: Use `view = a[2:5].copy()` if an independent copy is intended.

Debug 4

CODE

```
a = np.array([1,2,3,4,5,6])
b = a.reshape(4, 2) # ValueError!
```

Issue: 6 elements cannot be reshaped into a 4x2 = 8-element array; the total element count must match.

Fix: Use a shape whose product equals 6, e.g. `reshape(2, 3)` or `reshape(3, 2)`.

Debug 5

CODE

```
a = np.array([1, 2, 3], dtype=np.int32)
a[0] = 3.9
print(a[0]) # expected 3.9, got 3
```

Issue: The array's dtype is int32, so assigning a float truncates it rather than upgrading the array's dtype.

Fix: Create the array with a float dtype from the start, e.g. `dtype=np.float64`, if fractional values are needed.

13. Interview Questions

Thirty questions spanning beginner to advanced, useful both for interview preparation and as a self-assessment checklist after the training.

13.1 Beginner

Q1. What is NumPy and why is it used?

A: NumPy is Python's core library for numerical computing, providing the fast, memory-efficient ndarray and vectorized mathematical operations, used because it's dramatically faster and more memory-efficient than pure Python lists for numeric work.

Q2. How do you create a NumPy array from a Python list?

A: `np.array([1, 2, 3])`

Q3. What is the difference between a Python list and a NumPy array?

A: A list can hold mixed types and has per-element overhead; an array holds a single fixed dtype in contiguous memory, enabling vectorized, C-speed operations.

Q4. How do you find the shape of an array?

A: Using the `.shape` attribute.

Q5. What does `ndim` represent?

A: The number of dimensions (axes) of an array.

Q6. How do you generate an array of evenly spaced numbers?

A: `np.linspace(start, stop, num)` for a fixed count, or `np.arange(start, stop, step)` for a fixed step size.

Q7. What is the default dtype of an integer NumPy array?

A: Typically `int64` on most 64-bit systems (platform-dependent).

Q8. How do you access the last element of an array?

A: Using negative indexing: `a[-1]`.

Q9. How is slicing different from indexing?

A: Indexing selects a single element; slicing selects a contiguous (or strided) range and returns an array.

Q10. What library is NumPy typically imported as?

A: `import numpy as np`

13.2 Intermediate

Q11. What is broadcasting and when does it fail?

A: Broadcasting lets arrays of different but compatible shapes be combined element-wise by virtually stretching size-1 dimensions. It fails when, comparing shapes from the right, two dimensions differ and neither is 1.

Q12. What's the difference between a view and a copy?

A: A view shares the underlying memory buffer with the original array (changes propagate both ways); a copy is an independent array with its own memory.

Q13. Which indexing methods return views vs. copies?

A: Basic slicing returns views. Fancy indexing (array/list of indices) and boolean indexing always return copies.

Q14. What is a ufunc?

A: A universal function — a NumPy function that applies element-wise across arrays, implemented in compiled C for speed, and that supports broadcasting.

Q15. How do you stack arrays vertically vs horizontally?

A: `np.vstack()` stacks along a new/first axis (rows); `np.hstack()` stacks along the existing horizontal axis (columns).

Q16. What does `axis=0` vs `axis=1` mean for a 2D array?

A: `axis=0` operates down the rows (column-wise results); `axis=1` operates across the columns (row-wise results).

Q17. How do you find the indices that would sort an array?

A: `np.argsort(a)`

Q18. What is the difference between `np.dot()` and element-wise multiplication?

A: `np.dot()` (or `@`) performs true matrix/vector multiplication (sums of products); ``*`` multiplies corresponding elements independently.

Q19. How would you remove duplicate values from an array?

A: `np.unique(a)`

Q20. What does the `-1` mean in `reshape(-1, 1)`?

A: NumPy automatically computes that dimension's size so the total element count is preserved.

13.3 Advanced

Q21. How does NumPy achieve its speed advantage over pure Python?

A: By storing data in contiguous, fixed-type memory blocks and pushing element-wise loops into pre-compiled C code (vectorization), avoiding Python's per-element interpreter overhead.

Q22. Explain strides in the context of ndarray.

A: Strides are the number of bytes to step in memory to move one position along each axis; they let NumPy represent reshapes, transposes, and slices without copying data, by reinterpreting how the same buffer is walked.

Q23. How would you compute the inverse and determinant of a matrix?

A: `np.linalg.inv(A)` for the inverse; `np.linalg.det(A)` for the determinant.

Q24. What happens if you try to invert a singular matrix?

A: `np.linalg.inv()` raises a `LinAlgError` because a singular matrix (determinant 0) has no inverse.

Q25. How do eigenvalues and eigenvectors relate to a matrix?

A: They satisfy $A v = \lambda v$; `np.linalg.eig(A)` returns the eigenvalues λ and corresponding eigenvectors v that solve this equation.

Q26. How can you vectorize a custom function that isn't natively vectorized?

A: With `np.vectorize()`, though it's mainly a convenience wrapper and doesn't give C-level speed; true speed gains come from expressing the logic in native NumPy operations.

Q27. What's the memory advantage of using in-place operations (e.g. `a += b`) vs `a = a + b`?

A: In-place operations reuse the existing array's memory rather than allocating a new array, reducing peak memory use and allocation overhead, especially in loops.

Q28. How does NumPy handle mixed-dtype arithmetic (e.g. `int + float`)?

A: It applies type promotion rules (upcasting) so the result dtype can represent both operands without loss, e.g. `int64 + float64 -> float64`.

Q29. Why might `np.random.default_rng()` be preferred over `np.random.seed()`?

A: The Generator API is instance-based rather than relying on hidden global state, making random sequences safer to reason about, reproducible, and free of cross-contamination between unrelated code paths.

Q30. How would you efficiently compute pairwise distances between two sets of points without explicit loops?

A: By reshaping the two point sets so their dimensions broadcast against each other (e.g. shape `(n,1,d)` vs `(1,m,d)`), then computing norms along the last axis — letting broadcasting generate the full pairwise grid in one vectorized expression.

14. Best Practices

14.1 Memory Optimization

- Choose the smallest dtype that safely fits your data (e.g. int8 instead of int64 for small counts).
- Prefer in-place operations (`a += b`, `np.add(a, b, out=a)`) inside loops to avoid repeated allocation.
- Use views (slicing) instead of copies when you don't need independent data.
- Release large intermediate arrays with `del` when no longer needed in long-running scripts.

14.2 Vectorization

- Replace explicit for-loops over array elements with array-level expressions wherever possible.
- Use boolean masks instead of filtering with list comprehensions.
- Use built-in aggregate functions (`sum`, `mean`, `std`) rather than manual accumulation loops.

14.3 Broadcasting

- Design array shapes deliberately so broadcasting does the work instead of manual tiling (`np.tile / np.repeat`).
- Use `np.newaxis` (or `reshape`) to add a dimension explicitly when you need broadcasting along a specific axis.

14.4 Common Mistakes

Mistake	Why It's a Problem	Fix
Using <code>*</code> for matrix multiplication	Produces element-wise product, not matrix product	Use <code>@</code> or <code>np.dot()</code>
Modifying a slice unintentionally	Basic slices are views — edits affect the original array	Use <code>.copy()</code> when independence is needed
Ignoring dtype on assignment	Assigning a float into an int array silently truncates it	Choose the correct dtype up front
Looping in Python over array elements	Loses NumPy's C-level speed advantage entirely	Replace with vectorized operations
Forgetting axis in aggregate functions	Collapses the wrong dimension, giving a single scalar instead of per-row/column results	Explicitly pass <code>axis=0</code> or <code>axis=1</code>

14.5 Performance Tips

- Profile before optimizing — use `%timeit` in Jupyter or the `time` module to confirm bottlenecks.
- Prefer built-in NumPy functions over `np.vectorize()`, which is a convenience wrapper, not a true C-speed loop.
- Batch operations across whole arrays rather than looping row-by-row.

- For very large-scale numeric work beyond NumPy's single-machine design, consider libraries built on similar principles, such as Dask or CuPy.

© Shalinee Priya

15. Cheat Sheet — Quick Reference

Array Creation

Function	Description
<code>np.array(list)</code>	Create array from a Python list
<code>np.zeros(shape)</code>	Array of zeros
<code>np.ones(shape)</code>	Array of ones
<code>np.full(shape, val)</code>	Array filled with a constant
<code>np.eye(n)</code>	$n \times n$ identity matrix
<code>np.arange(start, stop, step)</code>	Evenly stepped values
<code>np.linspace(start, stop, num)</code>	Evenly spaced values, fixed count
<code>a.reshape(shape)</code>	Reshape without copying data
<code>a.flatten()</code> / <code>a.ravel()</code>	Flatten to 1D (copy / view)

Inspecting Arrays

Attribute	Description
<code>a.shape</code>	Tuple of dimension sizes
<code>a.ndim</code>	Number of dimensions
<code>a.size</code>	Total number of elements
<code>a.dtype</code>	Data type of elements
<code>a.astype(dtype)</code>	Cast to a new dtype

Mathematical Functions

Function	Description
<code>np.sum</code> / <code>np.mean</code> / <code>np.median</code>	Aggregate statistics
<code>np.std</code> / <code>np.var</code>	Spread statistics
<code>np.min</code> / <code>np.max</code> / <code>np.argmin</code> / <code>np.argmax</code>	Extremes and their positions
<code>np.percentile(a, q)</code>	q -th percentile
<code>np.sqrt</code> / <code>np.log</code> / <code>np.exp</code>	Element-wise transforms
<code>np.sin</code> / <code>np.cos</code> / <code>np.tan</code>	Trigonometric functions

Indexing & Slicing

Syntax	Description
<code>a[i]</code>	Element at index <code>i</code>
<code>a[i:j:k]</code>	Slice from <code>i</code> to <code>j</code> , step <code>k</code>
<code>a[-1]</code>	Last element
<code>a[a > x]</code>	Boolean filter
<code>a[[0, 2, 4]]</code>	Fancy indexing by index list

Broadcasting & Matrix Ops

Syntax	Description
<code>a * scalar</code>	Broadcast scalar across array
<code>A @ B / np.dot(A, B)</code>	Matrix multiplication
<code>A.T</code>	Transpose
<code>np.linalg.inv(A)</code>	Matrix inverse
<code>np.linalg.det(A)</code>	Determinant
<code>np.linalg.eig(A)</code>	Eigenvalues & eigenvectors

Random Module

Syntax	Description
<code>rng = np.random.default_rng(seed)</code>	Create a seeded generator
<code>rng.integers(low, high, size)</code>	Random integers
<code>rng.random(size)</code>	Random floats in <code>[0,1)</code>
<code>rng.normal(loc, scale, size)</code>	Normal distribution samples
<code>rng.uniform(low, high, size)</code>	Uniform distribution samples
<code>rng.shuffle(a)</code>	Shuffle in place
<code>rng.choice(a, size, replace)</code>	Random sample from array

16. Summary

This training session covered the full arc of practical NumPy usage in a single 3-hour session:

- NumPy's purpose and its performance/memory advantage over Python lists
- Creating, inspecting, reshaping, and flattening arrays of 1, 2, and 3 dimensions
- Element-wise arithmetic, comparisons, and universal functions
- Indexing, slicing, step slicing, fancy indexing, and boolean filtering
- Core statistical and mathematical functions
- Manipulating arrays: concatenation, stacking, splitting, sorting, and uniqueness
- Broadcasting rules and how they eliminate explicit loops
- Matrix operations and linear algebra: multiplication, transpose, inverse, determinant, eigenvalues
- Generating reproducible random data with the modern Generator API
- Why NumPy outperforms pure Python, quantified with a benchmark
- A complete, real-world Student Marks Analysis project built end-to-end
- Practice questions, coding exercises, debugging drills, and 30 interview questions
- Best practices for memory, vectorization, and avoiding common mistakes

i NOTE

Keep this manual as a working reference. The Cheat Sheet in Section 15 is designed to be consulted directly while coding, without needing to re-read the full manual.

Where to Go Next

- pandas — labeled, tabular data analysis built on top of NumPy
- Matplotlib / Seaborn — turning NumPy arrays into charts
- scikit-learn — machine learning, consuming NumPy arrays as its core data structure
- SciPy — advanced scientific computing extending NumPy's linear algebra and statistics